

CSED211: Bomb lab Report

컴퓨터공학과 20240940 장해영

1. Overview

총 6개의 phase에 있는 폭탄을 터뜨리지 않고 모두 빠져나오는 것이 목표이다. main함수를 disassemble해보면 각 phase의 명칭이 phase_n으로 되어있는 것을 확인할 수 있다. 또한 각 phase함수를 호출한 뒤 phase_defused 함수를 호출하여 defuse가 됨을 확인하는 것 같다. 아래 풀이에서 (register)는 address mode로, 해당 레지스터의 값을 주소로 가지는 값으로 사용하였다.

2. Phase_1

```
(gdb) disas phase_1
Dump of assembler code for function phase_1:
0x0000000000400ef0 <+0>: sub    $0x8,%rsp
0x0000000000400ef4 <+4>: mov    $0x402480,%esi
0x0000000000400ef9 <+9>: callq 0x4012be <strings_not_equal>
0x0000000000400efe <+14>: test   %eax,%eax
0x0000000000400f00 <+16>: je     0x400f07 <phase_1+23>
0x0000000000400f02 <+18>: callq 0x401524 <explode_bomb>
0x0000000000400f07 <+23>: add    $0x8,%rsp
0x0000000000400f0b <+27>: retq
End of assembler dump.
```

Phase_1을 disassemble하면 다음과 같다. <phase_1+9>의 strings_not_equal함수를 보면 인자로 %rdi, %rsi를 받는다. 아래 instruction들을 따라 내려가며 해석해보

면, %rdi와 %rsi의 값이 같으면 0을, 값이 다르면 1을 return한다. 따라서 우리가 입력해야 하는 값인 %rdi와 %rsi(=%esi)의 값이 동일해야 phase_1이 해체되므로 %esi에 해당하는 0x402480에 해당하는 값을 입력해주어야 한다. 아래 사진처럼 x/s를 이용하여 0x402480

```
(gdb) x/s 0x402480
0x402480: "All your base are belong to us."
```

에 해당하는 "All your base are belong to us."가 phase_1의 답이다.

3. Phase_2

```
(gdb) disas phase_2
Dump of assembler code for function phase_2:
0x0000000000400f0c <+0>: push   %rbp
0x0000000000400f0d <+1>: push   %rbx
0x0000000000400f10 <+2>: sub    $0x28,%rsp
0x0000000000400f12 <+6>: mov    %rsp,%rsi
0x0000000000400f15 <+9>: callq 0x40155a <read_six_numbers>
0x0000000000400f1a <+14>: cmpl   $0x1,(%rsi)
0x0000000000400f1b <+15>: je     0x400f40 <phase_2+52>
0x0000000000400f20 <+20>: callq 0x401524 <explode_bomb>
0x0000000000400f25 <+25>: jmp    0x400f40 <phase_2+52>
0x0000000000400f27 <+27>: mov    %rdi,%rax
0x0000000000400f2a <+30>: add    %eax,%eax
0x0000000000400f2c <+32>: cmp    %eax,(%rbx)
0x0000000000400f2e <+34>: je     0x400f55 <phase_2+41>
0x0000000000400f30 <+36>: callq 0x401524 <explode_bomb>
0x0000000000400f35 <+41>: add    $0x4,%rbx
0x0000000000400f38 <+44>: cmp    %rbp,%rbx
0x0000000000400f39 <+45>: jne    0x400f77 <phase_2+72>
0x0000000000400f3c <+48>: callq 0x400f4c <phase_2+64>
0x0000000000400f3e <+50>: jmp    0x400f77 <phase_2+72>
0x0000000000400f40 <+52>: lea    0x4(%rsp),%rbp
0x0000000000400f45 <+57>: lea    0x10(%rsp),%rbp
0x0000000000400f4a <+62>: jmp    0x400f77 <phase_2+72>
0x0000000000400f4c <+64>: add    $0x28,%rsp
0x0000000000400f50 <+68>: pop    %rbx
0x0000000000400f51 <+69>: pop    %rbp
0x0000000000400f52 <+70>: retq
End of assembler dump.
```

```
(gdb) disas read_six_numbers
Dump of assembler code for function read_six_numbers:
0x000000000040155a <+0>: sub    $0x18,%rsp
0x000000000040155e <+4>: mov    %rsi,%rdx
0x0000000000401561 <+7>: lea    0x4(%rsi),%rcx
0x0000000000401565 <+11>: lea    0x14(%rsi),%rax
0x0000000000401569 <+15>: mov    %rax,0x8(%rsp)
0x000000000040156e <+20>: lea    0x10(%rsi),%rax
0x0000000000401572 <+24>: mov    %rax,(%rsp)
0x0000000000401576 <+28>: lea    0xc(%rsi),%r9
0x000000000040157a <+32>: lea    0x8(%rsi),%r8
0x000000000040157e <+36>: mov    $0x4027a1,%esi
0x0000000000401583 <+41>: mov    $0x0,%eax
0x0000000000401588 <+46>: callq 0x400c30 <_isoc99_sscanf@plt>
0x000000000040158d <+51>: cmp    $0x5,%eax
0x0000000000401590 <+54>: jg     0x401597 <read_six_numbers+61>
0x0000000000401592 <+56>: callq 0x401524 <explode_bomb>
0x0000000000401597 <+61>: add    $0x18,%rsp
0x000000000040159b <+65>: retq
End of assembler dump.
```

Phase_2와 read_six_numbers를 disassemble하면 위와 같다. <read_six_numbers+51>를 보면 입력된 값의 개수인 %eax가 0x5보다 커야 폭탄을 피해 jump하는 것으로 보아 입력받아야 하는 인자는 6개인것으로 추정하였다. 또한 <phase_2+6>과 read_six_numbers를 보아 6개의 값이 (%rsp), (%rsp+0x4), (%rsp+0x8), (%rsp+0xc), (%rsp+0x10), (%rsp+0x14)에 각각 들어가는 것을 확인할 수 있다. 이러한 추정을 하고, 다시 phase_2함수로 돌아간다. 아래에 나오는 instruction번호는 모두 phase_2함수의 번호이다. 첫번째로 폭탄이 터지는 지점의 조건에 해당하는 <+14>을 보면 (%rsp)가 1이라면 폭탄을 피해 jump할 수 있으

므로 첫번째 값인 (%rsp)의 값은 1이 되어야 함을 알 수 있다. 다음으로, <+52>로 넘어가면 %rbx에는 (%rsp+0x4), %rbp에는 (%rsp+0x18)이 들어있는 상태로 다시 <+27>로 이동한후 <+27>~<+32>의 instruction을 확인하면 (%rsp) + (%rsp) 즉, (%rsp)를 두 배한 값과 (%rsp+0x4)의 값이 같으면 폭탄을 터뜨리지 않고 점프할 수 있다. 이렇게 두번째 값인 (%rsp+0x4)는 2로 추정하였다. 다음으로 <+41>을 보면, %rbx에 4를 더한다. 즉, 다음으로 입력된 값에 대해 값에 2를 곱하는 연산을 하게 된다. 이는 <+45>을 보면 알 수 있듯이 미리 (%rbp)에 저장해둔 마지막 값의 다음에 해당하는 값에 도달하기 전까지 반복한다. 즉, 등비가 2이고, 초항이 1이고, 항의 개수가 6개인 수열 "1 2 4 8 16 32"가 phase_2의 답이다.

4. Phase_3

```
(gdb) disas phase_3
Dump of assembler code for function phase_3:
0x0000000000400f53 <+0>: sub    $0x18,%rsp
0x0000000000400f57 <+4>: lea    0x8(%rsp),%rcx
0x0000000000400f5c <+9>: lea    0xc(%rsp),%rdx
0x0000000000400f61 <+14>: mov    $0x4027ad,%esi
0x0000000000400f66 <+19>: mov    $0x0,%eax
0x0000000000400f6b <+24>: callq  0x400c30 <_isoc99_sscanf@plt>
0x0000000000400f70 <+29>: cmp    $0x1,%eax
0x0000000000400f73 <+32>: jg     0x400f7a <phase_3+39>
0x0000000000400f75 <+34>: callq  0x401524 <explode_bomb>
0x0000000000400f7a <+39>: cmpl   $0x7,0xc(%rsp)
0x0000000000400f7f <+44>: ja     0x400fbd <phase_3+106>
0x0000000000400f81 <+46>: mov    0xc(%rsp),%eax
0x0000000000400f85 <+50>: jmpq   *0x4024e0(,%rax,8)
0x0000000000400f8c <+57>: mov    $0x13d,%eax
0x0000000000400f91 <+62>: jmp    0x400fce <phase_3+123>
0x0000000000400f93 <+64>: mov    $0x238,%eax
0x0000000000400f98 <+69>: jmp    0x400fce <phase_3+123>
0x0000000000400f9a <+71>: mov    $0x286,%eax
0x0000000000400f9f <+76>: jmp    0x400fce <phase_3+123>
0x0000000000400fa1 <+78>: mov    $0x29b,%eax
0x0000000000400fa6 <+83>: jmp    0x400fce <phase_3+123>
0x0000000000400fa8 <+85>: mov    $0x360,%eax
0x0000000000400fad <+90>: jmp    0x400fce <phase_3+123>
0x0000000000400faf <+92>: mov    $0xb3,%eax
0x0000000000400fb4 <+97>: jmp    0x400fce <phase_3+123>
0x0000000000400fb6 <+99>: mov    $0x1c5,%eax
0x0000000000400fbb <+104>: jmp    0x400fce <phase_3+123>
0x0000000000400fbd <+106>: callq  0x401524 <explode_bomb>
0x0000000000400fc2 <+111>: mov    $0x0,%eax
(gdb) x/8gx 0x4024e0
0x4024e0: 0x0000000000400f8c 0x0000000000400fc9
0x4024f0: 0x0000000000400f93 0x0000000000400f9a
0x402500: 0x0000000000400fa1 0x0000000000400fa8
0x402510: 0x0000000000400faf 0x0000000000400fb6
0x0000000000400fdd <+138>: retq
End of assembler dump.
```

Phase_3을 disassemble하면 다음과 같다. 첫번째로 폭탄이 있는 지점에 해당하는 조건<phase_3+29>를 보면 sscanf로 입력받은 값의 개수가 1개 이상이어야 함을 알 수 있다. 앞으로 나타내는 instruction의 위치는 phase_3 함수에서의 위치이다. <+14>의 0x4027ad에 있는 값을 보면 "%d %d"이다. 이를 통해 입력해야 하는 값은 정수 두 개임을 알았다. <+4>, <+9>에서 sscanf에서 입력받은 첫번째 값은 (%rsp+0xc)에, 두번째 값은 (%rsp+0x8)에 저장됨을 알 수 있다. 다음으로 <+39>를 보아

(%rsp+0xc)가 7보다 크면 폭탄이 터진다. 따라서 (%rsp+0xc)는 7이하 이다. <+50>에 jump table로 추측되는 주소가 있어 이 주소에 저장되어 있는 값을 확인하여 jump table을 찾은 것이 두번째 그림이다. (%rsp+0xc)의 값이 7이하 이므로 0~7을 고려하여 0x4024e0부터 8개의 값을 확인하였다. 다음으로는 <+129>에 있는 폭탄을 고려하면 <+123>에 있는 instruction을 이용하여 (%rsp+0x8)과 %eax가 같아야 폭탄이 터지지 않고 해체됨을 알 수 있다. 가능한 조합을 ((%rsp+0xc), (%rsp+0x8))의 순서쌍으로 나타내면 (0, 317), (1, 308), (2,568), (3, 646), (4, 667), (5, 864), (6, 179), (7, 453) 총 8개로 나타낼 수 있다. 다음단계로 넘어가기 위해 "0 317"을 입력하였다.

5. Phase_4

```
(gdb) disas phase_4
Dump of assembler code for function phase_4:
0x0000000000401011 <+0>: sub    $0x18,%rsp
0x0000000000401015 <+4>: lea    0x8(%rsp),%rcx
0x000000000040101a <+9>: lea    0xc(%rsp),%rdx
0x000000000040101f <+14>: mov    $0x4027ad,%esi
0x0000000000401024 <+19>: mov    $0x0,%eax
0x0000000000401029 <+24>: callq  0x400c30 <__isoc99_sscanf@plt>
0x000000000040102e <+29>: cmp    $0x2,%eax
0x0000000000401031 <+32>: jne     0x40103a <phase_4+41>
0x0000000000401033 <+34>: cmpl   $0xe,0xc(%rsp)
0x0000000000401038 <+39>: jbe     0x40103f <phase_4+46>
0x000000000040103a <+41>: callq  0x401524 <explode_bomb>
0x000000000040103f <+46>: mov    $0xe,%edx
0x0000000000401044 <+51>: mov    $0x0,%esi
0x0000000000401049 <+56>: mov    0xc(%rsp),%edi
0x000000000040104d <+60>: callq  0x400fde <func4>
0x0000000000401052 <+65>: cmp    $0x13,%eax
0x0000000000401055 <+68>: jne     0x40105e <phase_4+77>
0x0000000000401057 <+70>: cmpl   $0x13,0x8(%rsp)
0x000000000040105c <+75>: je      0x401063 <phase_4+82>
0x000000000040105e <+77>: callq  0x401524 <explode_bomb>
0x0000000000401063 <+82>: add    $0x18,%rsp
0x0000000000401067 <+86>: retq
End of assembler dump.

(gdb) disas func4
Dump of assembler code for function func4:
0x0000000000400fde <+0>: push   %rbx
0x0000000000400fdf <+1>: mov    %edx,%eax
0x0000000000400fe1 <+3>: sub    %esi,%eax
0x0000000000400fe3 <+5>: mov    %eax,%ebx
0x0000000000400fe5 <+7>: shr    $0x1f,%ebx
0x0000000000400fe8 <+10>: add    %ebx,%eax
0x0000000000400fea <+12>: sar    %eax
0x0000000000400fec <+14>: lea    (%rax,%rsi,1),%ebx
0x0000000000400fef <+17>: cmp    %edi,%ebx
0x0000000000400ff1 <+19>: jle     0x400fff <func4+33>
0x0000000000400ff3 <+21>: lea    -0x1(%rbx),%edx
0x0000000000400ff6 <+24>: callq  0x400fde <func4>
0x0000000000400ffb <+29>: add    %ebx,%eax
0x0000000000400ffd <+31>: jmp     0x40100f <func4+49>
0x0000000000400fff <+33>: mov    %ebx,%eax
0x0000000000401001 <+35>: cmp    %edi,%ebx
0x0000000000401003 <+37>: jge     0x40100f <func4+49>
0x0000000000401005 <+39>: lea    0x1(%rbx),%esi
0x0000000000401008 <+42>: callq  0x400fde <func4>
0x000000000040100d <+47>: add    %ebx,%eax
0x0000000000401010 <+50>: pop    %rbx
0x0000000000401010 <+50>: retq
End of assembler dump.
```

Phase_4와 func4를 disassemble하면 다음과 같다. 앞으로 나타내는 instruction의 위치는 phase_4 함수에서의 위치이다. <+14>의 0x4027ad에 있는 값을 보면 "%d %d"이다. 이를 통해 입력해야 하는 값은 정수 두 개임을 알았다. <+4>, <+9>에서 sscanf에서 입력받는 첫번째 값은 (%rsp+0xc)에, 두번째 값은 (%rsp+0x8)에 저장됨을 알 수 있다. 추가로, <+29>에서도 sscanf로 부터 입력받는 값의 개수가 2개임을 알 수 있다. <+34>~<+41>로부터 (%rsp+0xc)의 값이 0xe 즉, 14보다 작아야 폭탄이 터지지 않음을 추측할 수 있다. 다음으로 <+65>로 넘어가면 func4의 반환값과 0x13이 같아야 하고, <+70>에서는 (%rsp+0x8)도 0x13 즉, 19이어야 한다.

이제 disassemble한 func4에서 반환값이 어떻게 도출되는지 확인하자. 우선 <func4+46>~<func4+56>에서 알 수 있듯 func4의 인자는 순서대로 (%rsp+0xc), 0, 0xe로 총 3개이다. 앞으로 나타내는 instruction의 위치는 func4 함수에서의 위치이다. Func4의 instruction을 따라가며 수식을 세워보면 다음과 같다. $\{ \%edx - \%esi \} > 31(\text{logical}) + \{ \%edx - \%esi \} > 1(\text{arithmetic}) + \%rsi$, 이 값은 %ebx에 저장되고, %eax는 $\{ \%edx - \%esi \} > 31(\text{logical}) + \{ \%edx - \%esi \} > 1(\text{arithmetic})$ 의 값이다. 이 상태에서 jump의 조건에 따라 두가지의 갈래로 나뉜다.

- %edi(=%rdi = (%rsp+0xc))이 %ebx보다 작거나 같다면 <+33>으로 jump한다. %ebx값을 %eax에 옮기고 다시 위의 비교과정을 수행한다. 이 결과에 따라 다시 두 갈래로 나뉜다.

1. %edi(=%rdi = (%rsp+0xc))이 %ebx보다 크거나 같다면 %eax값 반환
2. %edi(=%rdi = (%rsp+0xc))이 %ebx보다 작다면 %esi를 %rbx+1로 바꾸고 다시 func4를 수행한다. 그 다음 이 반환값을 받아 %ebx를 더한 뒤 이 값을 반환한다.

- %edi(=%rdi = (%rsp+0xc))이 %ebx보다 크다면, %edx를 %rbx-1로 바꾸어 다시

func4를 수행한다. 그 다음 이 반환값을 받아 %ebx를 더한 뒤 이 값을 반환한다.

```
int func4(int edi, int esi, int edx) {
    int eax, ebx;
    eax = edx - esi;
    ebx = logicalShift(eax, 31); // logical
    eax = (eax + ebx) >> 1; // arithmetic
    ebx = eax + esi;

    if (ebx - edi <= 0) {
        // <+33>
        eax = ebx;
        if (ebx - edi >= 0) return eax;

        else {
            esi = ebx + 1;
            eax = func4(edi, esi, edx);
            eax += ebx;
            return eax;
        }
    }
    else {
        edx = ebx - 1;
        eax = func4(edi, esi, edx);
        eax += ebx;
        return eax;
    }
}
```

이 규칙을 알아보기 쉽게 C언어로 작성하면 다음과 같다.

이 규칙들을 이용하며 (%rsp+0xc)를 추론해야 한다. 우선 최종적으로 반환하는 값이 19가 나와야 한다는 것을 고려하자.

입력값인 (%rsp+0xc)를 4라 가정하고 if문이 나오기전 연산을 진행할 때 마다 (ebx, eax)를 나타내보면 (7, 7) -> (3, 3) -> (5, 1) -> (4, 0)이고, 이를 따라 연산을 진행하면 $4+5+3+7 = 19$ 로 값이 맞음을 확인할 수 있다.

6. Phase_5

```
Dump of assembler code for function phase_5:
0x0000000000401068 <+0>:    push    %rbx
0x0000000000401069 <+1>:    mov     %rdi,%rbx
0x000000000040106c <+4>:    callq  0x4012a1 <string_length>
0x0000000000401071 <+9>:    cmp     $0x6,%eax
0x0000000000401074 <+12>:   je      0x40107b <phase_5+19>
0x0000000000401076 <+14>:   callq  0x401524 <explode_bomb>
0x000000000040107b <+19>:   mov     $0x0,%eax
0x0000000000401080 <+24>:   mov     $0x0,%edx
0x0000000000401085 <+29>:   movzbl (%rbx,%rax,1),%ecx
0x0000000000401089 <+33>:   and     $0xf,%ecx
0x000000000040108c <+36>:   add     0x402520(,%rcx,4),%edx
0x0000000000401093 <+43>:   add     $0x1,%rax
0x0000000000401097 <+47>:   cmp     $0x6,%rax
0x000000000040109b <+51>:   jne     0x401085 <phase_5+29>
0x000000000040109d <+53>:   cmp     $0x41,%edx
0x00000000004010a0 <+56>:   je      0x4010a7 <phase_5+63>
0x00000000004010a2 <+58>:   callq  0x401524 <explode_bomb>
0x00000000004010a7 <+63>:   pop     %rbx
0x00000000004010a8 <+64>:   retq
End of assembler dump.
```

Phase_5를 disassemble하면 다음과 같다. 앞으로 나타내는 instruction의 위치는 phase_5 함수에서의 위치이다. <+4>에 있는 함수이름을 보아 문자열의 길이를 반환하는 함수인 것 같다. 다음 instruction을 보아 문자열의 길이가 6이 되어야 폭탄을 뛰어넘을 수 있으므로 입력해야 할 문자열의 길이는 6이다. <+19>와 <+24>에서 %eax

와 %edx의 값을 0으로 초기화한다. <+29>부터는 아래 <+51>에서 다시 돌아오는 instruction이 있으므로 반복문으로 파악할 수 있다. 그렇다면 이제 이 반복해야 할 내용이 무엇인지 알아보자. (%rbx + %rax)과 0xf를 and연산을 진행하여 %ecx에 저장한다. 이는 (%rbx + %rax)의 마지막 4bits만 남긴다는 의미로 해석할 수 있다. 그리고 $(4 * \%rcx + 0x402520)$ 을 %edx에 더한다. 그리고 %rax의 수를 하나 늘린다. 지금 까지가 반복에 해당하는 내용이다. 아래 <+47>~<+56>을 보아 %rax가 6, 이 때 %edx가 0x41 즉, 65이면 폭탄을 터뜨리지 않고 해체할 수 있다. %rax가 6이어야 반복이 종료된다는 것은, <+43>을 보아 총 6번 반복하는 것이라 해석할 수 있다. 6자리에 해당하는 값을 모두 더했을 때 65가 되어야 한다. 그 값을 추측하기 위해 0x402520에 해당하는 값을 확인하면 다음과

```
(gdb) x/16wu 0x402520
0x402520 <array.3161>:  2      10      6      1
0x402530 <array.3161+16>: 12     16     9      3
0x402540 <array.3161+32>: 4       7      14     5
0x402550 <array.3161+48>: 11      8      15     13
```

같다. 값을 16개까지 확인한 이유는 4bits로는 16종류를 나타낼 수 있기

때문이다. 또한 u로 옵션을 정하여 10진수로 나타내도록 하였다. 이제 이 값을 분석해보

자. %rcx의 값이 0이면 2, 1이면 12, ... , 15이면 13이 더해지는 형식이다. 이 값 중 6개를 더하여 65를 만들어야 하는데 가능한 조합은 (10, 7, 14, 11, 8, 15)이다. 즉, %rcx의 값에 16진수로 저장된다 가정하면 차례로 1, 9, a, c, d, e가 입력되어야 한다. 하위 4bits가 이 수들인 문자열을 아스키코드를 이용하여 찾으면 aijlmn이다. 이 때 이 문자열들의 아스키코드의 하위 4bits를 더하는 연산만 진행하므로 문자열의 순서는 상관없을 것으로 보인다. 다음단계로 넘어가기 위해 phase_5의 답은 "aijlmn"로 입력하였다.

7. Phase_6

```
(gdb) disas phase_6
Dump of assembler code for function phase_6:
0x00000000004010a9 <+0>:    push    %r13
0x00000000004010ab <+2>:    push    %r12
0x00000000004010ad <+4>:    push    %rbp
0x00000000004010ae <+5>:    push    %rbx
0x00000000004010af <+6>:    sub     $0x58,%rsp
0x00000000004010b3 <+10>:   lea     0x30(%rsp),%rsi
0x00000000004010b8 <+15>:   callq   0x40155a <read_six_numbers>
0x00000000004010bd <+20>:   lea     0x30(%rsp),%r13
0x00000000004010c2 <+25>:   mov     $0x0,%r12d
0x00000000004010c8 <+31>:   mov     %r13,%rbp
0x00000000004010cb <+34>:   mov     0x0(%r13),%eax
0x00000000004010cf <+38>:   sub     $0x1,%eax
0x00000000004010d2 <+41>:   cmp     $0x5,%eax
0x00000000004010d5 <+44>:   jbe     0x4010dc <phase_6+51>
0x00000000004010d7 <+46>:   callq   0x401524 <explode_bomb>
0x00000000004010dc <+51>:   add     $0x1,%r12d
0x00000000004010e0 <+55>:   cmp     $0x6,%r12d
0x00000000004010e4 <+59>:   jne     0x4010ed <phase_6+68>
0x00000000004010e6 <+61>:   mov     $0x0,%esi
0x00000000004010eb <+66>:   jmp     0x40112f <phase_6+134>
0x00000000004010ed <+68>:   mov     %r12d,%ebx
0x00000000004010f0 <+71>:   movslq   %ebx,%rax
0x00000000004010f3 <+74>:   mov     0x30(%rsp,%rax,4),%eax
0x00000000004010f7 <+78>:   cmp     %eax,0x0(%rbp)
0x00000000004010fa <+81>:   jne     0x401101 <phase_6+88>
0x00000000004010fc <+83>:   callq   0x401524 <explode_bomb>
0x0000000000401101 <+88>:   add     $0x1,%ebx
0x0000000000401104 <+91>:   cmp     $0x5,%ebx
0x0000000000401107 <+94>:   jle     0x4010f0 <phase_6+71>
0x0000000000401109 <+96>:   add     $0x4,%r13
0x000000000040110d <+100>:  jmp     0x4010c8 <phase_6+31>
0x000000000040110f <+102>:  mov     0x8(%rdx),%rdx
0x0000000000401113 <+106>:  add     $0x1,%eax
0x0000000000401116 <+109>:  cmp     %ecx,%eax
0x0000000000401118 <+111>:  jne     0x40110f <phase_6+102>
0x000000000040111a <+113>:  jmp     0x401121 <phase_6+120>
0x000000000040111c <+115>:  mov     $0x6042f0,%edx
0x0000000000401121 <+120>:  mov     %rdx,(&rsp,%rsi,2)
0x0000000000401125 <+124>:  add     $0x4,%rsi
0x0000000000401129 <+128>:  cmp     $0x18,%rsi
0x000000000040112d <+132>:  je      0x401144 <phase_6+155>
0x000000000040112f <+134>:  mov     0x30(%rsp,%rsi,1),%ecx
0x0000000000401133 <+138>:  cmp     $0x1,%ecx
0x0000000000401136 <+141>:  jle     0x40111c <phase_6+115>
0x0000000000401138 <+143>:  mov     $0x1,%eax
0x000000000040113d <+148>:  mov     $0x6042f0,%edx
0x0000000000401142 <+153>:  jmp     0x40110f <phase_6+102>
0x0000000000401144 <+155>:  mov     (&rsp),%rbx
0x0000000000401148 <+159>:  lea     0x8(%rsp),%rax
0x000000000040114d <+164>:  lea     0x30(%rsp),%rsi
```

```
0x0000000000401152 <+169>:  mov     %rbx,%rcx
0x0000000000401155 <+172>:  mov     (%rax),%rdx
0x0000000000401158 <+175>:  mov     %rdx,0x8(%rcx)
0x000000000040115c <+179>:  add     $0x8,%rax
0x0000000000401160 <+183>:  cmp     %rsi,%rax
0x0000000000401163 <+186>:  je      0x40116a <phase_6+193>
0x0000000000401165 <+188>:  mov     %rdx,%rcx
0x0000000000401168 <+191>:  jmp     0x401155 <phase_6+172>
0x000000000040116a <+193>:  movq    $0x0,0x8(%rdx)
0x0000000000401172 <+201>:  mov     $0x5,%ebp
0x0000000000401177 <+206>:  mov     0x8(%rbx),%rax
0x000000000040117b <+210>:  mov     (%rax),%eax
0x000000000040117d <+212>:  cmp     %eax,(&rbx)
0x000000000040117f <+214>:  jle     0x401186 <phase_6+221>
0x0000000000401181 <+216>:  callq   0x401524 <explode_bomb>
0x0000000000401186 <+221>:  mov     0x8(%rbx),%rbx
0x000000000040118a <+225>:  sub     $0x1,%ebp
0x000000000040118d <+228>:  jne     0x401177 <phase_6+206>
0x000000000040118f <+230>:  add     $0x58,%rsp
0x0000000000401193 <+234>:  pop     %rbx
0x0000000000401194 <+235>:  pop     %rbp
0x0000000000401195 <+236>:  pop     %r12
0x0000000000401197 <+238>:  pop     %r13
0x0000000000401199 <+240>:  retq
End of assembler dump.
```

Phase_6를 disassemble하면 위와 같다. 앞으로 나타내는 instruction의 위치는 phase_6 함수에서의 위치이다. 이 함수는 크게 6가지 파트로 나눌 수 있을 것 같다. 사진의 파란색 선으로 파트를 구분하였으며, 구분기준은 jmp로 무조건 점프하는 instruction으로 정하였다. 첫번째 파트부터 보자. <+15>에서 phase_2에서와 같은 함수인 read_six_numbers를 사용하고 있으므로 같은 사

고방식으로 (%rsp+0x30), (%rsp+0x34), ... , (%rsp+0x44)에 총 6개의 수가 각각 저장됨을 추론할 수 있다.

다음으로 <+20>~<+44>를 보면 첫번째로 입력받은 값은 %eax에 저장되고, 이 값에서 1 뺀 것이 5보다 작거나 같아야 한다. 즉, 입력값이 6보다 작아야 한다는 것이다. 다음으로 <+51>로 넘어가면 %r12d에 1을 더하고 다시 6과 비교한다. 이때 %r12d는 <+100>에서 다시 <+31>로 jump하는 instruction이 있는 것을 보아 반복수를 count하는 레지스터로 이해할 수 있을 것 같다. 6이 아니라면 <+68>로, 6이 맞다면 %esi에 0을 저장 후 <+134>로 점프한다.

이제 두번째 파트를 보자. 위의 분석에 따르면 이 파트는 count가 6이 아닌 경우에 해당한다. 즉, 남은 값들에 대해서 연산을 진행해야 한다는 것이다. <+68>~<+74>를 보

면 %eax에 다음 값이 저장됨을 확인할 수 있다. 이 다음 값과 현재 값이 달라야 폭탄을 피해 jump한다.

그 후 속 반복수라 생각할 수 있는 %ebx를 1증가시키고 이 수가 5보다 작거나 같다면 <+71>로 점프한다. 5보다 크다면 다음값을 분석하기 위해 다시 <+31>로 돌아가 반복한다. 이 때 <+71>에서는 위에서 분석했듯 다음값과 현재값을 비교하여 같으면 폭탄을 터뜨리고 같지 않다면 %ebx가 5가 넘을 때까지 이 과정을 반복한다. 새로운 수가 저장될 때 마다 지금까지 저장되어 있는 수와 중복이 있는 지 확인하는 과정으로 이해할 수 있다.

즉, 지금까지 첫번째 파트와 두번째 파트의 instruction을 요약하면, 6개의 정수를 입력받으며, 이 6개의 입력된 값이 모두 6보다 작고, 모두 다른 수여야 한다.

아직 세번째 파트에 있는 라인으로 점프하는 instruction이 나오지 않았으므로 위에서 등장한 <+134>가 있는 네번째 파트를 먼저 분석하는 것이 좋아보인다. 앞 instruction에서 6개의 수가 모두 중복되지 않는 6이하의 수임을 확인하면 <+134>로 점프하게 된다. %rsi의 초기값은 0임을 생각하고 네번째 파트를 해석하면 다음과 같다. (%rsp+%rsi+0x30)값 즉, 현재 분석하고 있는 입력된 값이 1보다 작으면 <+115>로 점프하고 그렇지 않으면 %eax에 1을, %edx에 0x6042f0을 저장한 후 <+102>로 점프한다.

<+115>에서는 %edx를 0x6042f0으로 저장하고 (%rsp + 2*%rsi)를 이 값으로 저장한다. 그 후 %rsi에 4를 더하여 다음 값으로 넘어간 후 위 <+134>에서 했던 과정을 다시 반복하고, 이 값이 마지막 입력값이면 <+155>로 jump한다. 즉, 위의 <+134>~ 를 6번 반복하는 것이다.

0x6042f0이 매우 의심스러워 이 주소에 있는 값을 찾아보면 6개의 node가 나오는 것을 확인할 수 있다.

0x6042f0 <node1>:	0x0000027e	0x00000001	0x00604300	0x00000000
0x604300 <node2>:	0x000001d0	0x00000002	0x00604310	0x00000000
0x604310 <node3>:	0x00000259	0x00000003	0x00604320	0x00000000
0x604320 <node4>:	0x0000036f	0x00000004	0x00604330	0x00000000
0x604330 <node5>:	0x00000123	0x00000005	0x00604340	0x00000000
0x604340 <node6>:	0x000001bc	0x00000006	0x00000000	0x00000000

노드 하나는 16bytes로 이루어져 있고, 첫번째 4bytes에는 값이, 두번째 4bytes에는 노드 번호가, 세번째 4bytes에는 다음 노드의 주소가 있는 것으로 추론할 수 있다.

즉, 네번째 파트에서는 (%rsp)부터 8bytes씩 띄어 (%rsp+0x8), ..., (%rsp+0x28)에 각각 입력한 수의 번호를 가진 노드의 주소가 순서대로 저장되는 것으로 이해할 수 있다.

아제 세번째 파트의 instruction을 분석해보자. 현재 입력된 값인 %ecx와 %eax가 같아질 때까지 %eax를 증가시키고 이에 따라 %rdx도 8씩 증가시켜 입력된 값에 해당하는 노드의 주소와 번호를 업데이트한다. 모든 업데이트가 완료되면, 다시 네번째 파트를 실행하여 총 6개의 노드의 주소를 (%rsp)부터 차례로 8bytes씩 띄어 저장하게 된다.

즉, 현재 %rsp를 기준으로 다음과 같이 저장되어 있다. 박스 아래에 적힌 수는 박스 안의 값이 (%rsp+n)일 때 n의 값을 16진수로 나타낸 것이다.

%rsp ->	1st 주소	2nd 주소	3rd 주소	4th 주소	5th 주소	6th 주소	1st 값	2nd 값	3rd 값	4th 값	5th 값	6th 값
(0x)0	8	10	18	20	28	30	34	38	3c	40	44	48

이제 다섯번째 파트를 분석해보자. 각종 레지스터를 이용하여 연산을 하는 것을 확인할 수 있다. 연산을 따라가보면, 위 노드 구성에서 각 노드의 3번째 4bytes에 다음 노드주소가 저장되어 있는 것을, 입력한 노드번호의 순서대로 다음노드를 지정하여 변경한다는 것을 확인할 수 있다.

이제 마지막 여섯번째 파트를 분석해보자. <+193>을 보면 마지막 노드는 연결할 노드가 없으므로 다음노드의 주소를 의미하는 %rdx+0x8을 주소로 하는 값에 0을 저장하는 것을 확인할 수 있다. 그리고 <+201>~<+216>을 보면 현재 노드에 있는 값이 다음노드에 있는 값보다 작은 경우만 폭탄을 터뜨리지 않고 jump한다. 그리고 이 과정을 6번 반복하므로 node에 저장되어 있는 값을 기준으로 오름차순으로 노드를 정렬해야 함을 알 수 있다.

이제 다시 node의 정보를 보면, 1번노드에 있는 값은 0x17e, 2번노드에 있는 값은 0x1d0, 3번노드에 있는 값은 0x259, 4번노드에 있는 값은 0x36f, 5번노드에 있는 값은 0x123, 6번노드에 있는 값은 0x1bc이므로 이 값을 기준으로 노드를 오름차순으로 정렬하면 5, 6, 2, 3, 1, 4 이고, phase_6의 답은 "5 6 2 3 1 4"이다.

```
(gdb) r
Starting program: /home/std/hyjang/bomb14/bomb
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
All your base are belong to us.
Phase 1 defused. How about the next one?
1 2 4 8 16 32
That's number 2. Keep going!
0 317
Halfway there!
4 19
So you got that one. Try this one.
aijlmn
Good work! On to the next...
5 6 2 3 1 4
Congratulations! You've defused the bomb!
Your instructor has been notified and will verify your solution.
```

이렇게 모든 phase를 해결할 수 있다.

8. Secret_phase

과제 안내서에서 secret point가 있음을 확인하고, main함수에서 secret phase를 찾으려 했으나, 존재하지 않아 위 overview에서 확인하였던 phase_defused함수를 disassemble해보았다.

```
(gdb) disas phase_defused
Dump of assembler code for function phase_defused:
0x00000000004016c2 <+0>: sub    $0x68,%rsp
0x00000000004016c6 <+4>: mov    $0x1,%edi
0x00000000004016cb <+9>: callq 0x401460 <send_msg>
0x00000000004016d0 <+14>: cmpl   $0x6,0x2030c5(%rip) # 0x60479c <num_input_strings>
0x00000000004016d7 <+21>: jne    0x401746 <phase_defused+132>
0x00000000004016d9 <+23>: lea    0x10(%rsp),%r8
0x00000000004016de <+28>: lea    0x8(%rsp),%rcx
0x00000000004016e3 <+33>: lea    0xc(%rsp),%rdx
0x00000000004016e8 <+38>: mov    $0x4027f7,%esi
0x00000000004016ed <+43>: mov    $0x6048b0,%edi
0x00000000004016f2 <+48>: mov    $0x0,%eax
0x00000000004016f7 <+53>: callq 0x400c30 <__isoc99_sscanf@plt>
0x00000000004016fc <+58>: cmp    $0x3,%eax
0x00000000004016ff <+61>: jne    0x401732 <phase_defused+112>
0x0000000000401701 <+63>: mov    $0x402800,%esi
0x0000000000401706 <+68>: lea    0x10(%rsp),%rdi
0x000000000040170b <+73>: callq 0x4012be <strings_not_equal>
0x0000000000401710 <+78>: test   %eax,%eax
0x0000000000401712 <+80>: jne    0x401732 <phase_defused+112>
0x0000000000401714 <+82>: mov    $0x402658,%edi
0x0000000000401719 <+87>: callq 0x400b40 <puts@plt>
0x000000000040171e <+92>: mov    $0x402680,%edi
0x0000000000401723 <+97>: callq 0x400b40 <puts@plt>
0x0000000000401728 <+102>: mov    $0x0,%eax
0x000000000040172d <+107>: callq 0x4011d8 <secret_phase>
0x0000000000401732 <+112>: mov    $0x4026b8,%edi
0x0000000000401737 <+117>: callq 0x400b40 <puts@plt>
0x000000000040173c <+122>: mov    $0x4026e8,%edi
0x0000000000401741 <+127>: callq 0x400b40 <puts@plt>
0x0000000000401746 <+132>: add    $0x68,%rsp
0x000000000040174a <+136>: retq
End of assembler dump.
```

<phase_defused+107>에 secret_phase가 있음을 확인하였다. 이 함수를 실행하기 위해서는 <phase_defused+107>이후의 line으로 jump하지 않아야 하므로 sscanf로 입력받은 값의 개수가 3개이면 된다. 또한 0x4027f7에 들어있는 문자열을 확인하면 와 같아 정수 두

```
(gdb) x/s 0x4027f7
0x4027f7: "%d %d %s"
```

개와 문자열 하나를 입력하면 된다는 것을 알았다. 그리고 이 문자열에는 0x402800에

```
(gdb) x/s 0x402800
0x402800: "DrJisungPark"
```

있는 문자열을 입력해야 jump하지 않으므로 이 문자열을 조사하면 "DrJisungPark"임을 확

인하였다. 그렇다면 이 정수 두 개와 문자열을 어디에 입력해야할까? 0x6048b0에 어떤 값이 저장되는 지 확인해봐야 한다. 이 값을 확인해보면 "4 19"로 phase_4의 입력값임을 알

```
(gdb) x/s 0x6048b0
0x6048b0 <input_strings+240>: "4 19"
```

수 있다. 그렇다면 phase_4의 답을 입력할

때 DrJisungPark를 추가로 입력하여 "4 19 DrJisungPark"을 입력한다면 secret_phase로 들

```
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
All your base are belong to us.
Phase 1 defused. How about the next one?
1 2 4 8 16 32
That's number 2. Keep going!
0 317
Halfway there!
4 19 DrJisungPark
So you got that one. Try this one.
aijlmn
Good work! On to the next...
5 6 2 3 1 4
Curses, you've found the secret phase!
But finding it and solving it are quite different...
█
```

어갈 수 있을 것으로 예상할 수 있다.

Phase_4에서 "format에 맞는 입력값"이 2개여야 폭탄이 터지지 않는데, 이 문자열이 추가되더라도 정수라는 format과 맞지 않으므로 폭탄이 터질 염려는 하지 않아도 될 것 같다. 실제로 실행해보면 옆 사진처럼 secret phase에 정상적으로

진입하였음을 확인하였다. 이제 secret_phase 함수를 disassemble하여 답을 찾아내도록 하자.

Secret_phase와 fun7을 disassemble하면 다음과 같다.

```
(gdb) disas secret_phase
Dump of assembler code for function secret_phase:
0x00000000004011d8 <+0>: push %rbx
0x00000000004011d9 <+1>: callq 0x40159c <read_line>
0x00000000004011de <+6>: mov $0xa,%edx
0x00000000004011e3 <+11>: mov $0x0,%esi
0x00000000004011e8 <+16>: mov %rax,%rdi
0x00000000004011eb <+19>: callq 0x400c00 <strtol@plt>
0x00000000004011f0 <+24>: mov %rax,%rbx
0x00000000004011f3 <+27>: lea -0x1(%rax),%eax
0x00000000004011f6 <+30>: cmp $0x3e8,%eax
0x00000000004011fb <+35>: jbe 0x401202 <secret_phase+42>
0x00000000004011fd <+37>: callq 0x401524 <explode_bomb>
0x0000000000401202 <+42>: mov %ebx,%esi
0x0000000000401204 <+44>: mov $0x604110,%edi
0x0000000000401209 <+49>: callq 0x40119a <fun7>
0x000000000040120e <+54>: cmp $0x3,%eax
0x0000000000401211 <+57>: je 0x401218 <secret_phase+64>
0x0000000000401213 <+59>: callq 0x401524 <explode_bomb>
0x0000000000401218 <+64>: mov $0x4024a0,%edi
0x000000000040121d <+69>: callq 0x400b40 <puts@plt>
0x0000000000401222 <+74>: callq 0x4016c2 <phase_defused>
0x0000000000401227 <+79>: pop %rbx
0x0000000000401228 <+80>: retq
End of assembler dump.

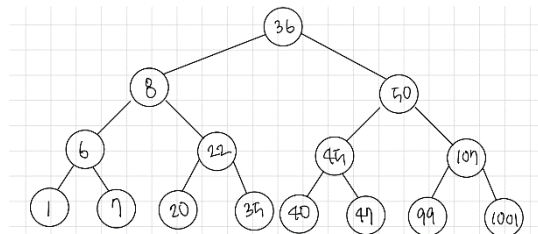
(gdb) disas fun7
Dump of assembler code for function fun7:
0x000000000040119a <+0>: sub $0x8,%rsp
0x000000000040119e <+4>: test %rdi,%rdi
0x00000000004011a1 <+7>: je 0x4011ce <fun7+52>
0x00000000004011a3 <+9>: mov (%rdi),%edx
0x00000000004011a5 <+11>: cmp %esi,%edx
0x00000000004011a7 <+13>: jle 0x4011b6 <fun7+28>
0x00000000004011a9 <+15>: mov 0x8(%rdi),%rdi
0x00000000004011ad <+19>: callq 0x40119a <fun7>
0x00000000004011b2 <+24>: add %eax,%eax
0x00000000004011b4 <+26>: jmp 0x4011d3 <fun7+57>
0x00000000004011b6 <+28>: mov $0x0,%eax
0x00000000004011bb <+33>: cmp %esi,%edx
0x00000000004011bd <+35>: je 0x4011d3 <fun7+57>
0x00000000004011bf <+37>: mov 0x10(%rdi),%rdi
0x00000000004011c3 <+41>: callq 0x40119a <fun7>
0x00000000004011c8 <+46>: lea 0x1(%rax,%rax,1),%eax
0x00000000004011cc <+50>: jmp 0x4011d3 <fun7+57>
0x00000000004011ce <+52>: mov $0xffffffff,%eax
0x00000000004011d3 <+57>: add $0x8,%rsp
0x00000000004011d7 <+61>: retq
End of assembler dump.
```

먼저 secret_phase를 보면 입력값을 %rbx에 저장하고, %rax-1을 %eax에 저장한다. 이 때 이 %eax의 값이 0x3e8보다 작아야 폭탄을 피할 수 있다. 즉, 입력값은 10001보다 작아야 한다. 그리고, fun7(0x604110, 입력값)을 한 결과가 3이어야 phase를 풀 수 있다. 이제 fun7을 통해 반환값이 3이 되도록 하기 위한 입력값을 찾도록 하자. 앞으로 나타내는 instruction의 위치는 fun7 함수에서의 위치이다.

- Fun7을 보면, 제일 먼저 %rdi가 0이면 -1을 반환한다.
- (%rdi)가 입력값보다 작거나 같으면 <+28>로 점프한다.
- (%rdi)가 더 크다면 (%rdi+0x8)를 %rdi에 저장하여 값을 업데이트한다. 그리고 fun7을 재귀적으로 호출한다. 그리고 이 반환값에 2를 곱하고 <+57>로 점프한다.
- 이제 <+28>을 보자. %eax에 0을 저장하고, %edx 즉, (%rdi)와 입력값을 비교한 후 같다면 <+57>로 점프한다.
- 같지 않다면 (%rdi+0x10)을 %rdi에 저장하여 값을 업데이트 한 뒤 fun7을 재귀적으로 호출한다. 그리고 %eax에 2*%rax+1를 저장하고 <+57>로 점프한다.

이 때 <+57>은 값을 return하는 instruction으로 이어진다. 위 규칙을 보면 BST에서 노드를 찾는 과정이 떠오른다. 2번 규칙은 left child node로 이동하는 것이고, 4번규칙은 right child node로 이동하는 것이다. 이 노드의 구성을 보고 tree를 그리면 다음과 같다.

```
0x604110 <n1>: 0x0000000000000024 0x00000000000004130
0x604120 <n1+16>: 0x000000000000064150 0x00000000000000000
0x604130 <n21>: 0x00000000000000008 0x0000000000000641b0
0x604140 <n21+16>: 0x000000000000064170 0x00000000000000000
0x604150 <n22>: 0x00000000000000032 0x000000000000064190
0x604160 <n22+16>: 0x0000000000000641d0 0x00000000000000000
0x604170 <n32>: 0x00000000000000016 0x000000000000064290
0x604180 <n32+16>: 0x000000000000064250 0x00000000000000000
0x604190 <n33>: 0x0000000000000002d 0x0000000000000641f0
0x6041a0 <n33+16>: 0x0000000000000642b0 0x00000000000000000
0x6041b0 <n31>: 0x00000000000000006 0x000000000000064210
0x6041c0 <n31+16>: 0x000000000000064270 0x00000000000000000
0x6041d0 <n34>: 0x0000000000000006b 0x000000000000064230
0x6041e0 <n34+16>: 0x0000000000000642d0 0x00000000000000000
0x6041f0 <n45>: 0x00000000000000028 0x00000000000000000
0x604200 <n45+16>: 0x00000000000000000 0x00000000000000000
0x604210 <n41>: 0x00000000000000001 0x00000000000000000
0x604220 <n41+16>: 0x00000000000000000 0x00000000000000000
0x604230 <n47>: 0x00000000000000063 0x00000000000000000
0x604240 <n47+16>: 0x00000000000000000 0x00000000000000000
0x604250 <n44>: 0x00000000000000023 0x00000000000000000
0x604260 <n44+16>: 0x00000000000000000 0x00000000000000000
0x604270 <n42>: 0x00000000000000007 0x00000000000000000
0x604280 <n42+16>: 0x00000000000000000 0x00000000000000000
0x604290 <n43>: 0x00000000000000014 0x00000000000000000
0x6042a0 <n43+16>: 0x00000000000000000 0x00000000000000000
0x6042b0 <n46>: 0x0000000000000002f 0x00000000000000000
0x6042c0 <n46+16>: 0x00000000000000000 0x00000000000000000
0x6042d0 <n48>: 0x000000000000003e9 0x00000000000000000
0x6042e0 <n48+16>: 0x00000000000000000 0x00000000000000000
```



이제 최종적으로 %eax가 3이 되도록 해보자. $3 == 2 * 1 + 1$, $1 == 1 * 0 + 1$ 이므로 %eax == 0에서 시작하여 right child node로 이동하는 실행을 2번하면 %eax == 3이 된다. 즉, 36으로부터 right child node로 두 번 이동하면 107이므로 secret_phase의 정답은 "107"이다.

```
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
All your base are belong to us.
Phase 1 defused. How about the next one?
1 2 4 8 16 32
That's number 2. Keep going!
0 317
Halfway there!
4 19 DrJisungPark
So you got that one. Try this one.
aijlmn
Good work! On to the next...
5 6 2 3 1 4
Curses, you've found the secret phase!
But finding it and solving it are quite different...
107
Wow! You've defused the secret stage!
Congratulations! You've defused the bomb!
Your instructor has been notified and will verify your solution.
```

옆 사진처럼 secret_phase까지 잘 실행됨을 확인하였다.

9. Result

"폭탄해체"라는 테마를 가지고 총 7개의 phase를 disassemble하여 코드를 추측하며 조건에 맞는 답을 찾았다. 이론만 공부하였을 때는 뜬구름잡는 내용인 것 같았지만, 이렇게 실제 어셈블리어를 해석하며 문제를 해결함으로써 어셈블리어에 훨씬 친숙해질 수 있는 과제였다.